Lab3: Leukemia Classification

312554029 陳映璇

1. Introduction

In this lab, I implemented ResNet18, ResNet50, and ResNet152 to classify acute lymphoblastic leukemia. For the DataLoader, I applied several data augmentation skills such as CenterCrop, Transpose, etc. to the training data. After training, I visualized the accuracy of the models by plotting the confusion matrix and accuracy curves to compare their performance.

2. Implementation Details

A. Details of the model (ResNet)

- ResNet (Residual Network) is a neural network architecture that enhances the training of deep neural networks by utilizing “skip connections”, which allow information from previous layers to bypass certain layers and directly contribute to later layers. This mitigates the vanishing gradient problem and facilitates training deeper networks.
- The following is the implementation of ResNet:
  - First, I define two class BasicBlock and Bottleneck for two different types of building blocks used to construct the network layers.

```python
class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride = 1):
        super().__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.stride = stride

        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size = 3, stride = stride, padding = 1, bias = False)
        self.bn1 = nn.BatchNorm2d(out_channels, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
        self.relu = nn.ReLU(inplace = True)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size = 3, stride = 1, padding = 1, bias = False)
        self.bn2 = nn.BatchNorm2d(out_channels, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)

        if in_channels != out_channels:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size = 1, stride = stride, bias=False),
                nn.BatchNorm2d(out_channels, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
            )

    def forward(self, x):
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)

        if self.in_channels != self.out_channels:
            out += self.downsample(x)
        else:
            out += x

        out = self.relu(out)
        return out
```

```python
class Bottleneck(nn.Module):
    def __init__(self, in_channels, mid_channels, out_channels, stride = 1):
        super().__init__()
        self.in_channels = in_channels
        self.mid_channels = mid_channels
        self.out_channels = out_channels
        self.stride = stride

        self.conv1 = nn.Conv2d(in_channels, mid_channels, kernel_size = 1, stride = 1, bias = False)
        self.bn1 = nn.BatchNorm2d(mid_channels, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
        self.conv2 = nn.Conv2d(mid_channels, mid_channels, kernel_size = 3, stride = stride, padding = 1, bias = False)
        self.bn2 = nn.BatchNorm2d(mid_channels, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
        self.conv3 = nn.Conv2d(mid_channels, out_channels, kernel_size = 1, stride = 1, bias = False)
        self.bn3 = nn.BatchNorm2d(out_channels, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
        self.relu = nn.ReLU(inplace = True)

        if in_channels != out_channels:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size = 1, stride = stride, bias = False),
                nn.BatchNorm2d(out_channels, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
            )
    def forward(self, x):
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)
        out = self.relu(out)
        out = self.conv3(out)
        out = self.bn3(out)

        if self.in_channels != self.out_channels:
            out += self.downsample(x)
        else:
            out += x

        out = self.relu(out)
        return out
```

- After that, I build ResNet18 using Basic Blocks and ResNet50 and ResNet152 using Bottleneck Blocks. The numbers in their names indicate the total number of layers in the network.

a. ResNet18

- Depth: 18 layers
- Basic Blocks: Utilizes the basic residual block with two 3x3 convolutions per block.

b. ResNet50

- Depth: 50 layers
- Bottleneck Blocks: Introduces the bottleneck architecture with 1x1, 3x3, and 1x1 convolutions in each block.

c. ResNet152

- Depth: 152 layers
- Bottleneck Blocks: Similar to ResNet50, it uses the bottleneck architecture with even more layers.

B. Details of the Dataloader

- The figure below is how I implemented to have a custom dataset:

```
class LeukemiaLoader(Dataset):
    def __init__(self, df, mode, transform = None):
        self.df = df
        self.mode = mode
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, index):
        image_path = self.df['Path'][index]
        image = cv2.imread(image_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        image = self.transform(image = image)['image']

        if self.mode != "test":
            label = self.df['label'][index]
            return image, label

        return image

train_df = pd.read_csv("/home/pp037/DL/Lab3/train.csv")
valid_df = pd.read_csv("/home/pp037/DL/Lab3/valid.csv")
```

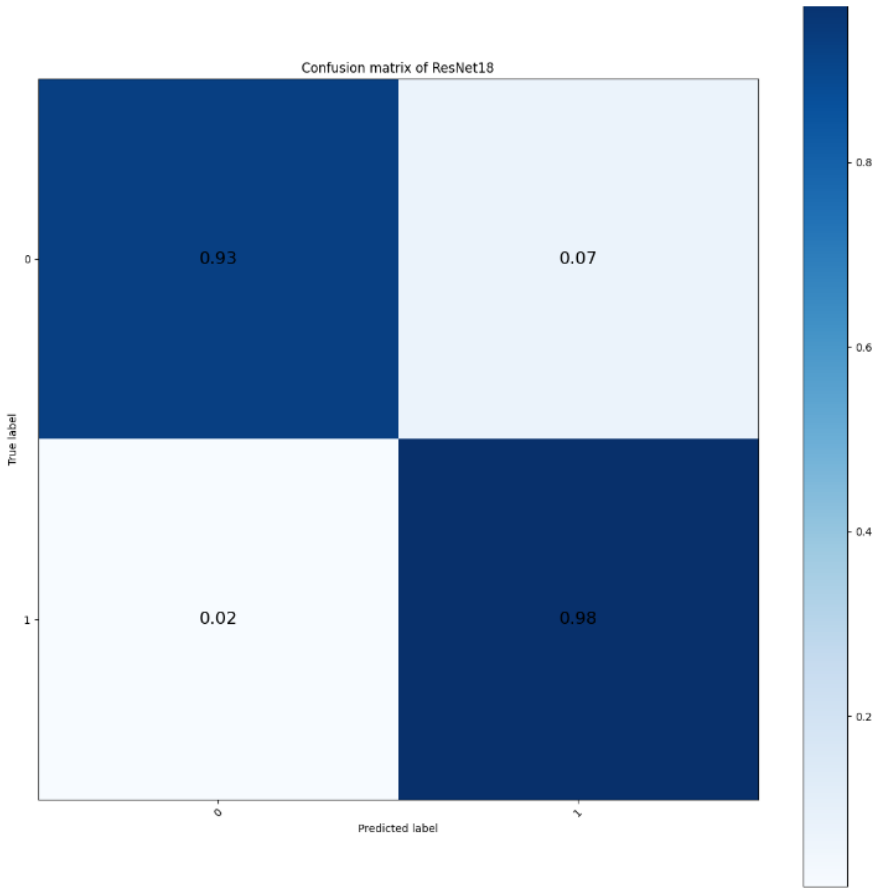
- I extract the image paths from the Dataframe, read the images

using OpenCV, and then convert the image from BGR to RGB format.

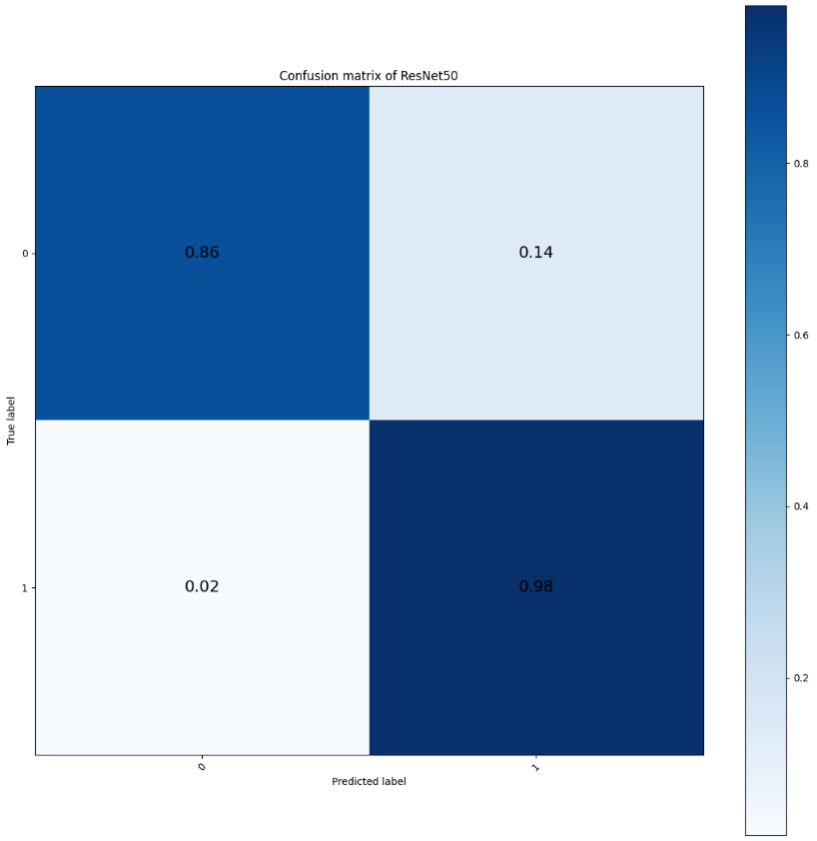
- For more on data augmentation, please refer to Section 3: Data preprocessing.

C. Evaluation through the confusion matrix

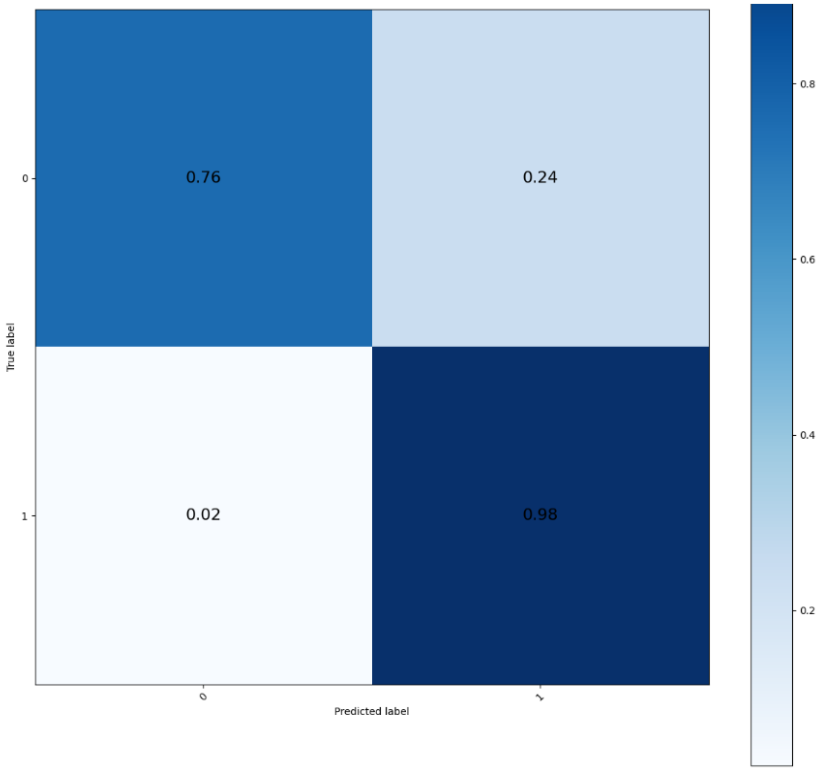
a. ResNet18



b. ResNet50



c. ResNet152



- From the above three confusion matrices, we can see that all three models predict label 1 well. As for the prediction of label 0, it seems that ResNet18 predicts with the highest accuracy. I think this may be due to its lighter network architecture, which is suitable for classifying relatively simple features. On the other hand, ResNet152 has a relatively low accuracy in predicting label 0 despite its higher number of layers and complex architecture, which may be attributed to the overfitting complex features and not performing as well as other models in simpler scenarios.

3. Data preprocessing

A. How to preprocess the data

```
import albumentations as A
from albumentations.pytorch import ToTensorV2

train_transform = A.Compose([
    A.CenterCrop(350, 350, p = 1.0),
    A.HorizontalFlip(p = 0.5),
    A.VerticalFlip(p = 0.5),
    A.ShiftScaleRotate(p = 0.5),
    A.CoarseDropout(p = 0.5),
    A.Rotate(p = 0.5),
    A.Transpose(p = 0.5),
    A.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225], max_pixel_value = 255, p = 1.0),
    ToTensorV2(p = 1.0)
])

valid_transform = A.Compose([
    A.CenterCrop(350, 350, p = 1.0),
    A.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225], max_pixel_value = 255, p = 1.0),
    ToTensorV2(p = 1.0)
])
```

- In addition to the Dataloader details mentioned in Section 2, I also import the *Albumentations* library for the purpose of applying some augmentation to the training data and valid data.
    - CenterCrop(350, 350, p = 1.0): Performs a center crop on the image, extracting a square region of size 350x350 pixels from the center. The probability of applying this transformation is 100% (p = 1.0).
    - A.HorizontalFlip(p = 0.5): Horizontally flips the image with a probability of 50% (p = 0.5). This creates a mirror image of the original along the vertical axis.
    - A.VerticalFlip(p = 0.5): Vertically flips the image with a probability of 50% (p = 0.5). This creates a mirror image of the original along the horizontal axis.

- A.ShiftScaleRotate(p = 0.5): Applies random shifts, scales, and rotations to the image with a probability of 50% (p = 0.5). This helps introduce spatial variations in the training data.
- A.CoarseDropout(p = 0.5): Randomly masks out rectangular regions of the image with a probability of 50% (p = 0.5). This simulates missing or occluded portions of the image.
- A.Rotate(p = 0.5): Rotates the image by a random angle with a probability of 50% (p = 0.5). This adds rotational diversity to the training data.
- A.Transpose(p = 0.5): Transposes the image (swaps rows and columns) with a probability of 50% (p = 0.5). This introduces additional geometric variations.
- A.Normalize(...): Applies image normalization by subtracting specified mean values and dividing by specified standard deviation values. This ensures consistent pixel value ranges for each channel.
- ToTensorV2(p = 1.0): Converts the image and its transformations into a PyTorch tensor. The probability of applying this transformation is 100% (p = 1.0).

B. What makes my method special?

- **Flexible Data Handling**: My dataset class has a __init__ method that takes in a DataFrame, a mode (such as "train", "valid", or "test"), and an optional data transformation. This flexibility in initialization allows us to use the same class for different modes of operation.

- **Data Transformation**: I use the Albumentations library to apply a series of image transformations to augment and preprocess data. Data augmentation helps in increasing the diversity of the training data, which can improve the generalization of the trained model.

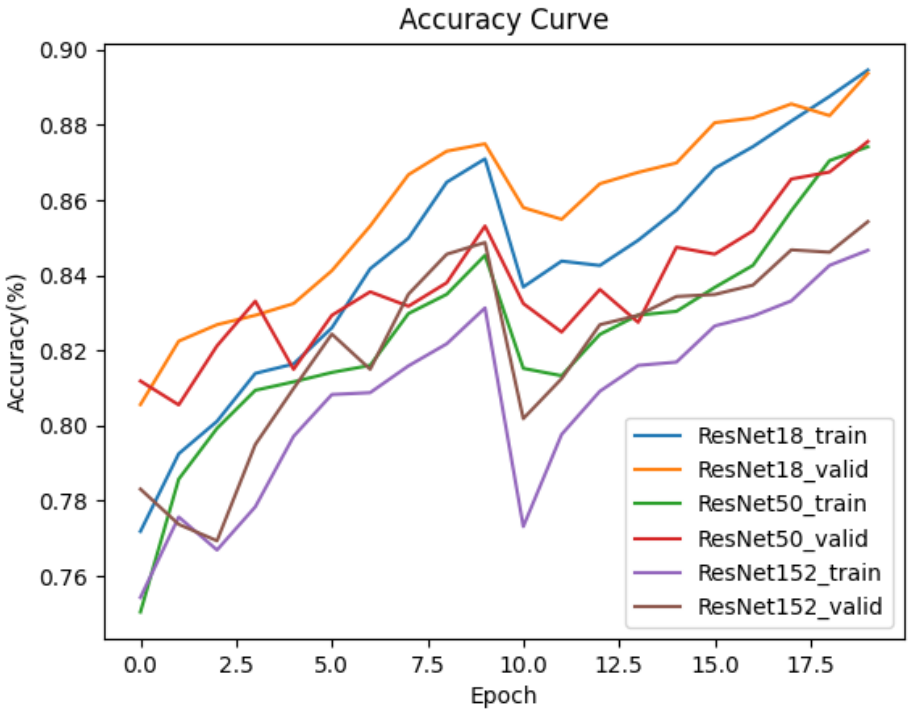
- **Efficient Data Loading**: The __getitem__ method is responsible for loading and preprocessing a single data sample based on the provided index. This method efficiently loads the image, applies transformations, and returns the preprocessed image along with its label (if not in test mode).

4. Experimental results

A. The highest testing accuracy

Model	ResNet18	ResNet50	ResNet152
Accuracy	97.469%	95.782%	91.940%

B. Comparison figures



5. Discussion

A. What I did to improve the accuracy:

- **Try various data augmentation techniques**: I experimented with different data augmentation techniques, including rotations, scaling, cropping, and flipping. This diversified the training dataset and helped prevent overfitting by enabling the model to generalize better from augmented examples.
- **Increase the batch size**: I raised the batch size, which involves

using more training examples in each optimization iteration. This decision contributed to more stable convergence as the model's parameters update were based on a more representative sample of the data.

- **Increase the epoch count**: I opted to increase the number of training epochs. This allowed the model to encounter the training data more times, refining its weights and potentially boosting accuracy over time.
- **Utilize an optimizer**: I employed a suitable optimizer AdamW to adjust the weights of the neural network during training. This choice speed up convergence and potentially led to higher accuracy.
- **Apply a scheduler**: I integrated a scheduler into the training process. Initially, a higher learning rate expedited convergence, while gradually decreasing it allowed the model to fine-tune its parameters for improved accuracy.

B. What are the differences between ResNet18, ResNet50, and ResNet152?

- **Depth and Number of Layers**:
  a. ResNet18: It has 18 layers, including convolutional layers, batch normalization, activation functions, and a few fully connected layers at the end.
  b. ResNet50: It has 50 layers and is deeper than ResNet18. It's divided into blocks with residual connections, which helps mitigate the vanishing gradient problem.
  c. ResNet152: It is the deepest among the three, with 152 layers. Its increased depth enables it to learn more complex features but also requires more computational resources for training and inference.
- **Model Complexity**:

a. ResNet18 is the simplest in terms of model complexity and is suitable for tasks where computational resources are limited.

b. ResNet50 strikes a balance between complexity and performance, making it a widely used choice for various computer vision tasks.

c. ResNet152 is the most complex and can capture very intricate features, but it requires significantly more computational resources